

Retail Sales Data Analysis Using Python

Capstone Project

Student Name: Carlos Brown Mwalim Murithi

Student ID: 26/00708

Course: Data Analytics

Institution: KCA University

Date: July 2026

Project Objective

The purpose of this project is to analyze retail sales data, clean the dataset, perform exploratory data analysis, visualize trends, develop a machine learning model for revenue prediction, and provide business recommendations.

✓ Step 1: Import Required Libraries

This section imports all Python libraries required throughout the project.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
```

```
)  
  
import warnings  
warnings.filterwarnings("ignore")  
  
np.random.seed(42)  
  
# Libraries imported successfully.
```

✓ Step 2: Load the Dataset

The dataset is loaded into a pandas DataFrame.

```
df = pd.read_csv("Capstone_Project_data.csv")  
  
df.head()
```

	date	region	product_category	units_sold	unit_price	revenue	discount_pct	
0	1/1/2022	East	Electronics	132	486.59	64275.21	0.073	
1	1/1/2022	West	Electronics	151	107.96	16305.57	0.120	
2	1/2/2022	North	Electronics	260	352.29	91698.29	0.006	
3	1/3/2022	East	Electronics	167	111.12	18485.12	0.232	
4	1/4/2022	East	Electronics	147	237.70	34916.14	NaN	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.tail()
```



	date	region	product_category	units_sold	unit_price	revenue	discount_pct
1195	12/27/2024	North	Electronics	177	316.84	56154.87	NaN
1196	12/28/2024	South	Food	197	25.73	5032.07	0.052
1197	12/29/2024	East	Electronics	202	371.88	75049.80	NaN
1198	12/30/2024	East	Electronics	242	318.30	76920.95	0.123
1199	12/31/2024	North	Electronics	87	414.42	36053.00	0.105

The dataset contains retail sales information including:

- Date
- Region
- Product Category
- Units Sold
- Unit Price
- Revenue
- Discount Percentage

✓ Step 3: Data Exploration

The objective is to understand the structure and quality of the dataset.

```
print(df.shape)
```

```
(1200, 7)
```

```
df.dtypes
```

```
0
```

	0
date	object
region	object
product_category	object
units_sold	int64
unit_price	float64
revenue	float64
discount_pct	float64

dtype: object

```
df.isnull().sum()
```

```
0
```

	0
date	0
region	0
product_category	0
units_sold	0
unit_price	0
revenue	0
discount_pct	98

dtype: int64

The only missing values are found in the Discount Percentage column.

```
df.describe()
```

	units_sold	unit_price	revenue	discount_pct
count	1200.000000	1200.000000	1200.000000	1102.000000
mean	201.144167	145.080200	29230.499875	0.150418
std	49.383219	130.218505	28012.393258	0.085981
min	49.000000	5.060000	614.710000	0.000000
25%	167.750000	39.807500	7780.015000	0.075000
50%	200.000000	103.080000	19758.420000	0.148500
75%	234.000000	215.930000	42058.722500	0.225750
max	359.000000	499.530000	171982.260000	0.300000



The statistical summary provides the minimum, maximum, average, and distribution of numerical variables.

✓ Step 4: Data Cleaning

Preparing the data before analysis.

```
median_discount = df["discount_pct"].median()

df["discount_pct"] = df["discount_pct"].fillna(median_discount)
```

Missing discount values are replaced using the median because it is less affected by outliers.

```
df["date"] = pd.to_datetime(df["date"])
#converting Date
```

```
df["year"] = df["date"].dt.year
#extracting year
```

```
df["month"] = df["date"].dt.month
```

```
df["net_revenue"] = df["revenue"] * (1 - df["discount_pct"])
```

Net revenue represents the revenue remaining after discounts have been applied.

```
df_encoded = pd.get_dummies(  
    df,  
    columns=["region", "product_category"],  
    drop_first=True  
)
```

Machine learning models require numerical inputs, so categorical variables are converted into dummy variables.

✓ Step 5: Exploratory Data Analysis

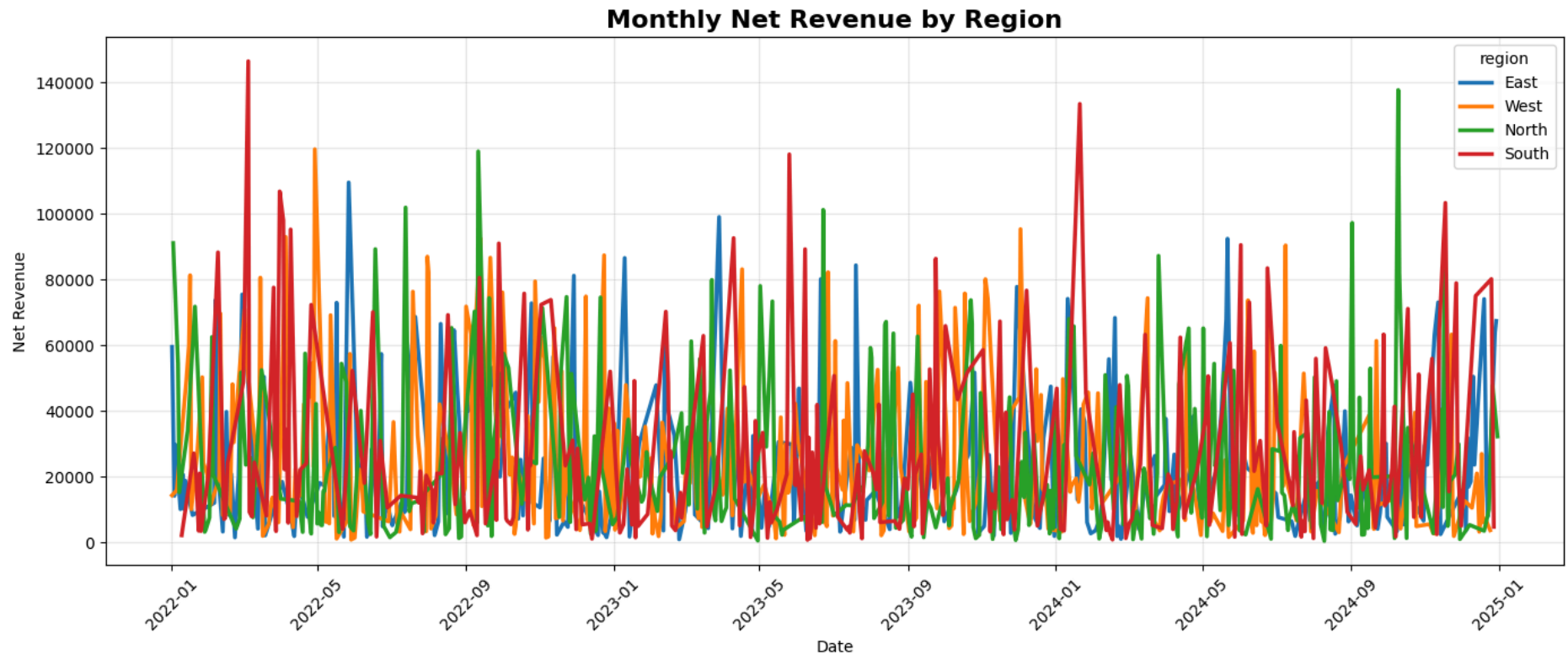
Visualizing sales patterns and trends.

✓ Monthly Net Revenue by Region

```
# Create monthly revenue table  
monthly = df.groupby(['date', 'region'])['net_revenue'].sum().reset_index()  
  
# Plot  
plt.figure(figsize=(14,6))  
  
sns.lineplot(  
    data=monthly,  
    x='date',  
    y='net_revenue',  
    hue='region',  
    linewidth=2.5  
)
```

```
plt.title("Monthly Net Revenue by Region", fontsize=16, fontweight='bold')
plt.xlabel("Date")
plt.ylabel("Net Revenue")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)
plt.tight_layout()

plt.show()
```



Interpretation

- Revenue fluctuates over time.
- Some regions consistently outperform others.

- The business can identify high-performing regions for expansion.

✓ Chart 2: Average Units Sold by Product Category

Objective

To compare the average number of units sold across different product categories.

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=df,
    x='product_category',
    y='units_sold',
    palette='viridis'
)

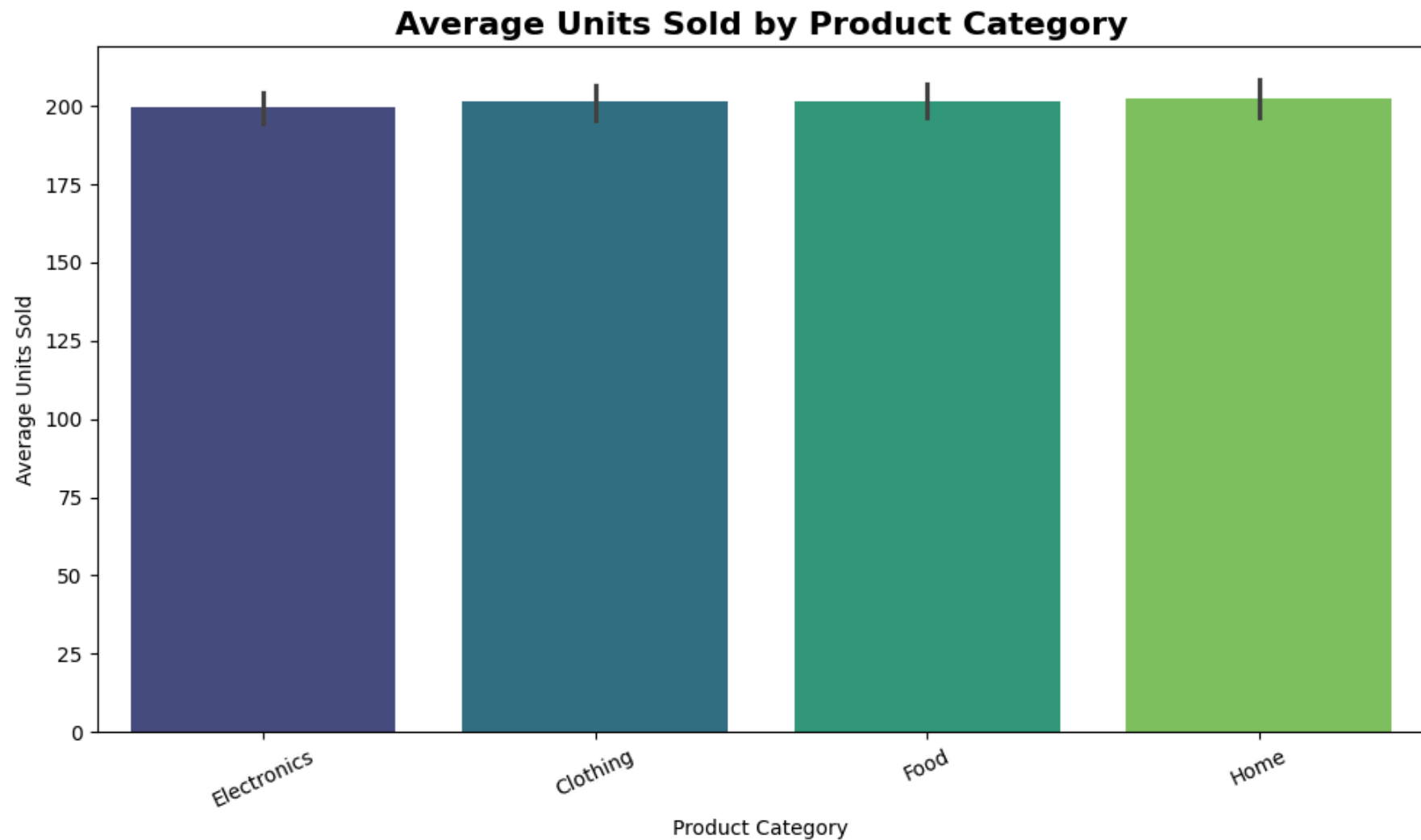
plt.title("Average Units Sold by Product Category",
          fontsize=16,
          fontweight='bold')

plt.xlabel("Product Category")
plt.ylabel("Average Units Sold")

plt.xticks(rotation=25)

plt.tight_layout()

plt.show()
```

Interpretation

- Some product categories sell significantly more units than others.
- High-selling categories should receive higher inventory levels.

Chart 3: Revenue Distribution by Region

Objective

To compare revenue variation across regions and identify possible outliers.

```
plt.figure(figsize=(10,6))

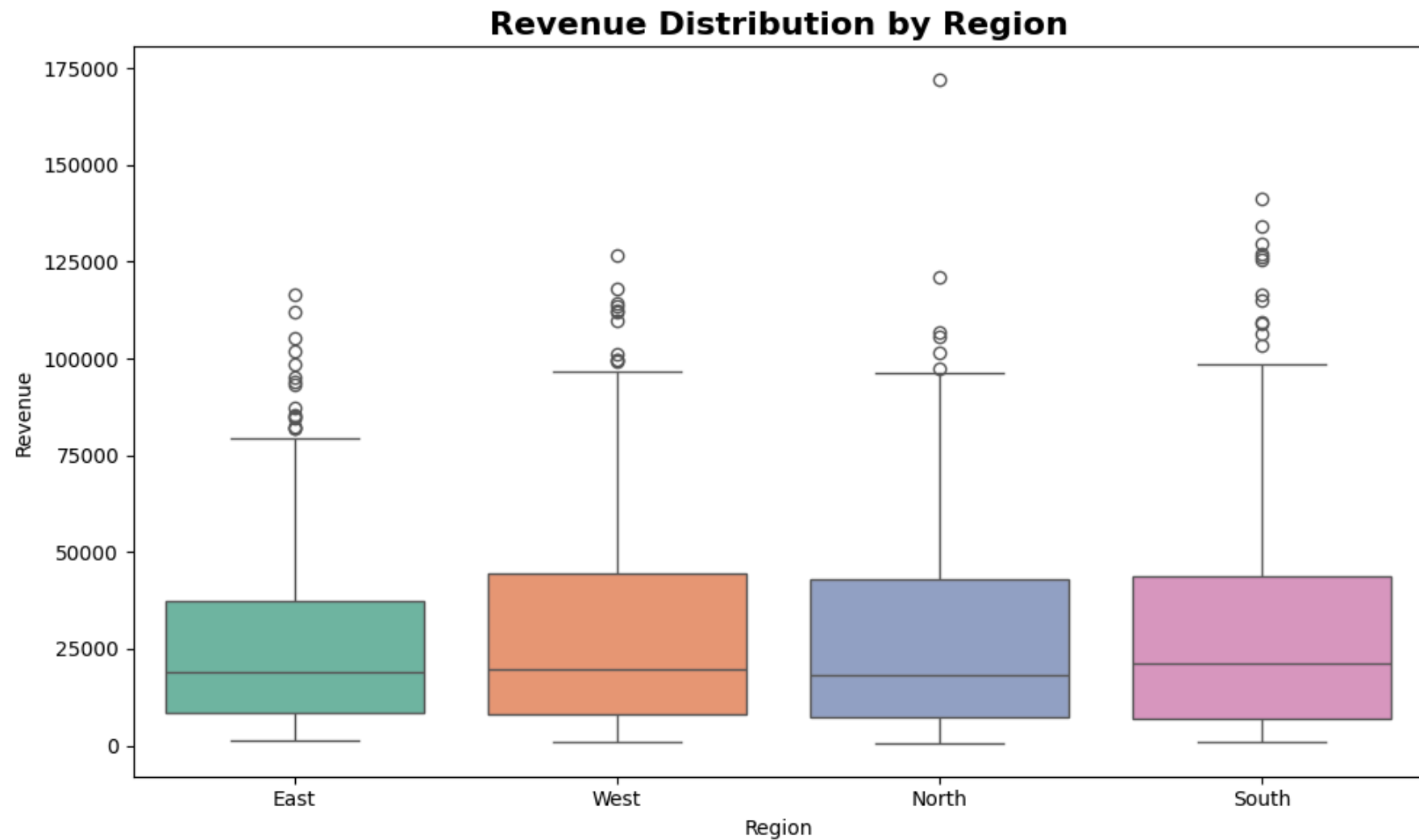
sns.boxplot(
    data=df,
    x='region',
    y='revenue',
    palette='Set2'
)

plt.title(
    "Revenue Distribution by Region",
    fontsize=16,
    fontweight='bold'
)

plt.xlabel("Region")
plt.ylabel("Revenue")

plt.tight_layout()

plt.show()
```



Interpretation

- Some regions have wider revenue variation.
- Outliers indicate exceptionally high sales transactions.



Chart 4: Relationship Between Units Sold and Net Revenue

Objective

To determine whether higher sales volumes generate higher revenue.

```
plt.figure(figsize=(10,6))

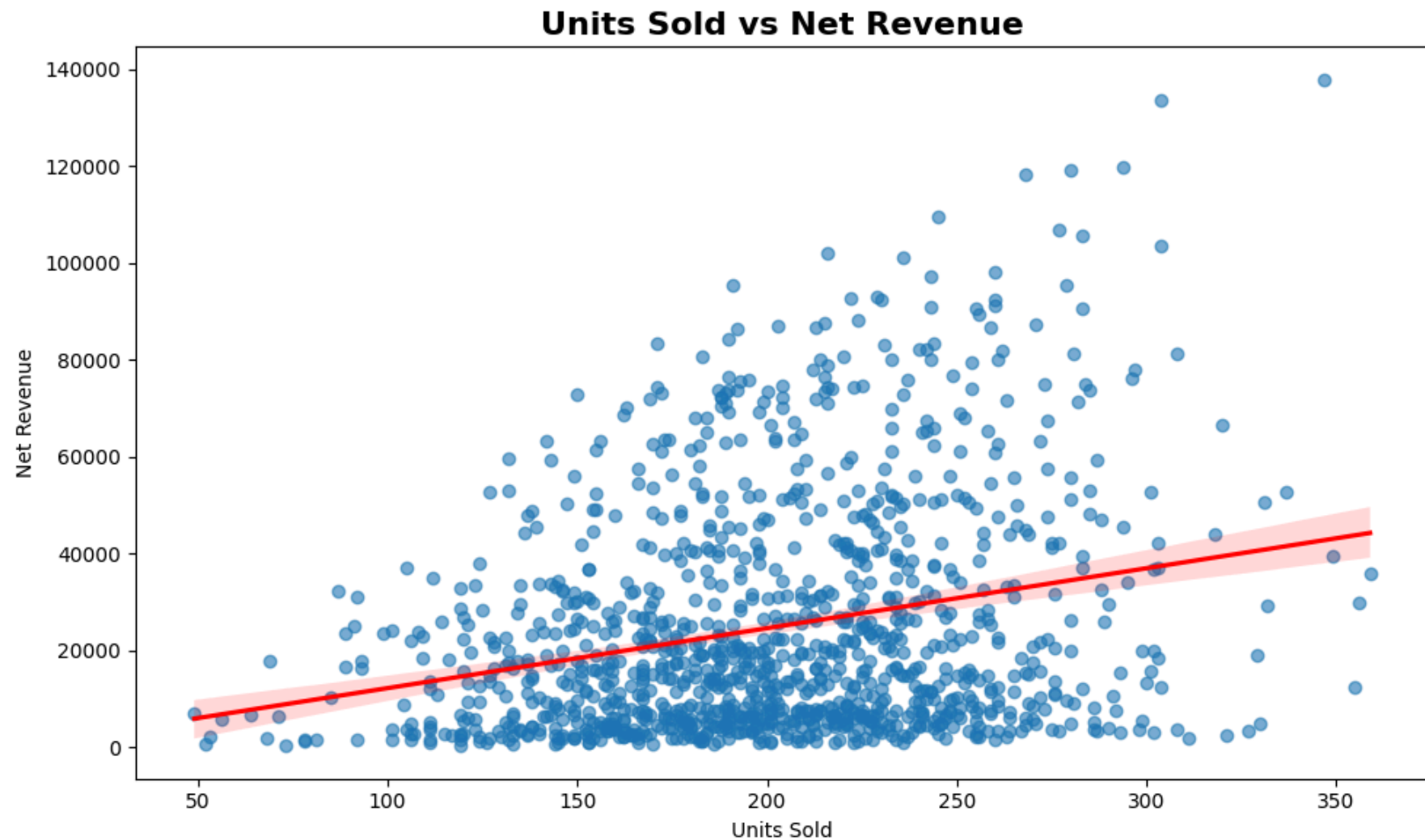
sns.regplot(
    data=df,
    x='units_sold',
    y='net_revenue',
    scatter_kws={'alpha':0.6},
    line_kws={'color':'red'}
)

plt.title(
    "Units Sold vs Net Revenue",
    fontsize=16,
    fontweight='bold'
)

plt.xlabel("Units Sold")
plt.ylabel("Net Revenue")

plt.tight_layout()

plt.show()
```



Interpretation

- There is a positive relationship between units sold and net revenue.
- Increasing sales volume generally increases business revenue.



Chart 5: Correlation Heatmap

Objective

To examine relationships between numerical variables in the dataset.

```
plt.figure(figsize=(12,8))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap='coolwarm',
    linewidths=0.5
)

plt.title(
    "Correlation Matrix",
    fontsize=16,
    fontweight='bold'
)

plt.tight_layout()

plt.show()
```


Correlation Matrix

Interpretation

- Strong positive correlations indicate variables that increase together.
- Strong negative correlations indicate variables moving in opposite directions.
- This chart helps identify important predictors for machine learning.

Chart 6: Total Revenue by Product Category

Objective

To determine which product category contributes the most revenue.

```
category = df.groupby('product_category')['net_revenue'].sum().reset_index()
```

```
plt.figure(figsize=(10,6))
```

```
sns.barplot(
    data=category,
    x='product_category',
    y='net_revenue',
    palette='magma'
)
```

```
plt.title(
    "Total Net Revenue by Product Category",
    fontsize=16,
    fontweight='bold'
)
```

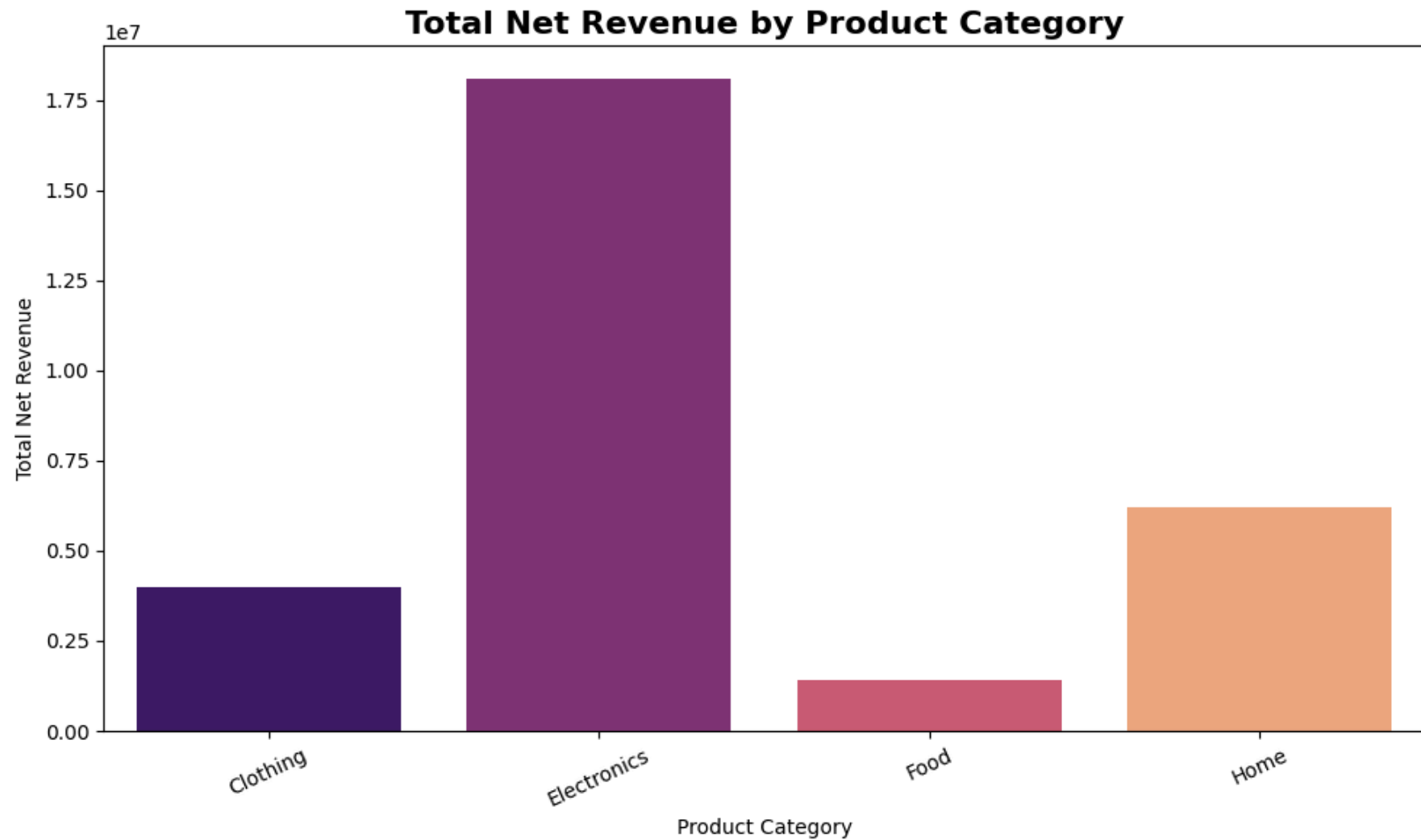
```
plt.xlabel("Product Category")
plt.ylabel("Total Net Revenue")
```

```
plt.xticks(rotation=25)
```



```
plt.tight_layout()
```

```
plt.show()
```



✓ Interpretation

- The chart identifies the product category generating the highest revenue.

- Management can prioritize these products in future marketing campaigns.### Interpretation
- The chart identifies the product category generating the highest revenue.
- Management can prioritize these products in future marketing campaigns.

```
#Selecting FFeatures
features = [
    "units_sold",
    "unit_price",
    "discount_pct",
    "year",
    "month"
] + [
    c for c in df_encoded.columns
    if "region_" in c or "product_" in c
]

X = df_encoded[features]

y = df_encoded["net_revenue"]
```

```
#Splitting Data
X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
#Train Model
model = LinearRegression()

model.fit(X_train,y_train)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```
#Prediction
y_pred = model.predict(X_test)
```

✓ Step 7: Model Evaluation

```
print("R² =", r2_score(y_test,y_pred))
print("MAE =", mean_absolute_error(y_test,y_pred))
print("RMSE =", np.sqrt(mean_squared_error(y_test,y_pred)))
```

```
R² = 0.9525219676706098
MAE = 3430.7935862081863
RMSE = 4962.405863394387
```

A high R^2 score indicates that the model explains most of the variation in net revenue.

Lower MAE and RMSE values indicate better prediction accuracy.

Actual vs Predicted Values

✓ Step 8: Actual vs Predicted Values

Objective

After training the Linear Regression model, it is important to evaluate how accurately it predicts Net Revenue.

One of the best ways to do this is by comparing the actual values from the dataset with the values predicted by the model.

If the predictions closely match the actual values, the model is considered accurate.

```
plt.figure(figsize=(10,7))

plt.scatter(
    y_test,
```

```
        y_pred,
        alpha=0.7,
        s=60
    )

    # Perfect prediction line
    plt.plot(
        [y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()],
        color='red',
        linewidth=2,
        linestyle='--'
    )

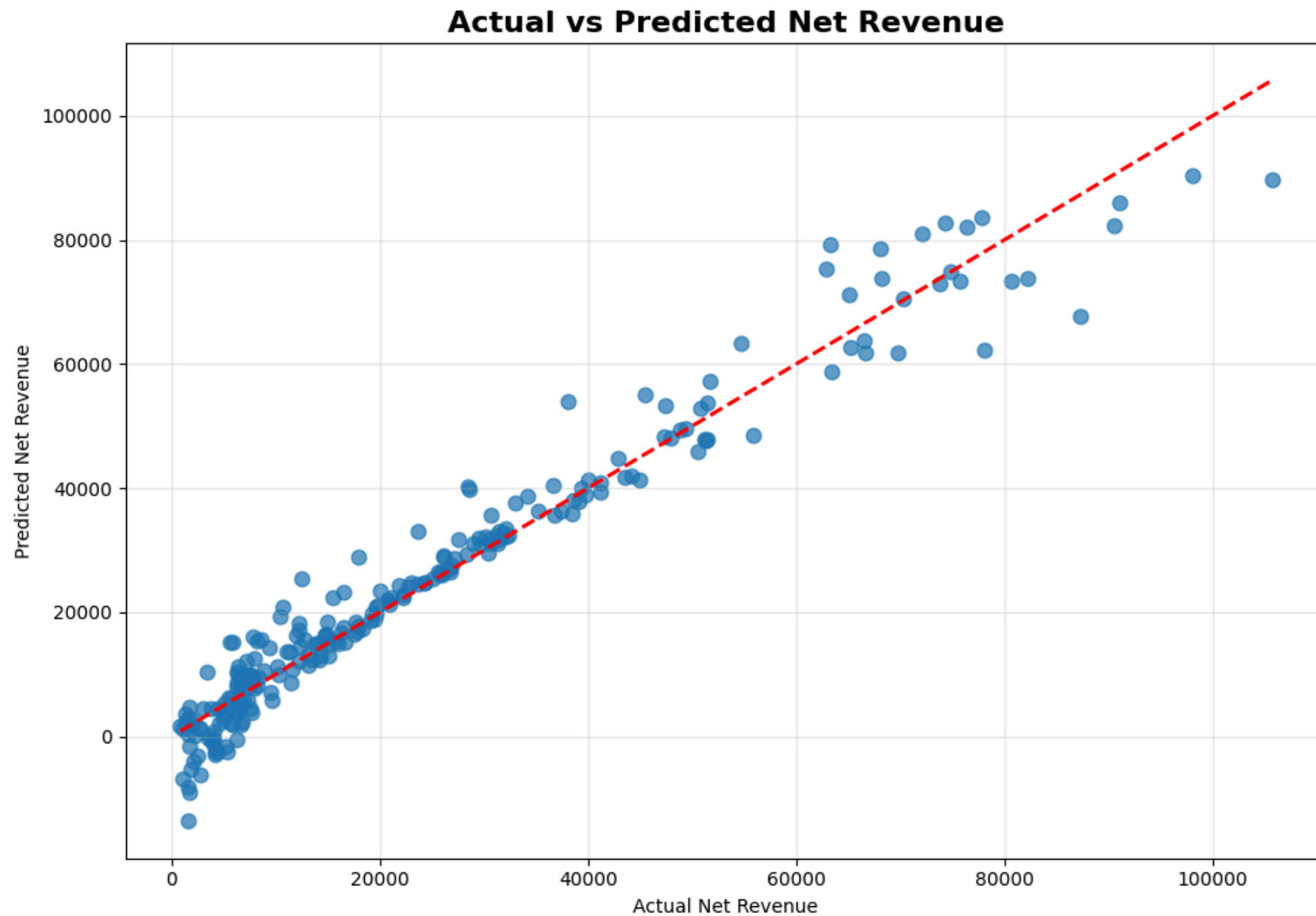
    plt.title(
        "Actual vs Predicted Net Revenue",
        fontsize=16,
        fontweight='bold'
    )

    plt.xlabel("Actual Net Revenue")
    plt.ylabel("Predicted Net Revenue")

    plt.grid(alpha=0.3)

    plt.tight_layout()

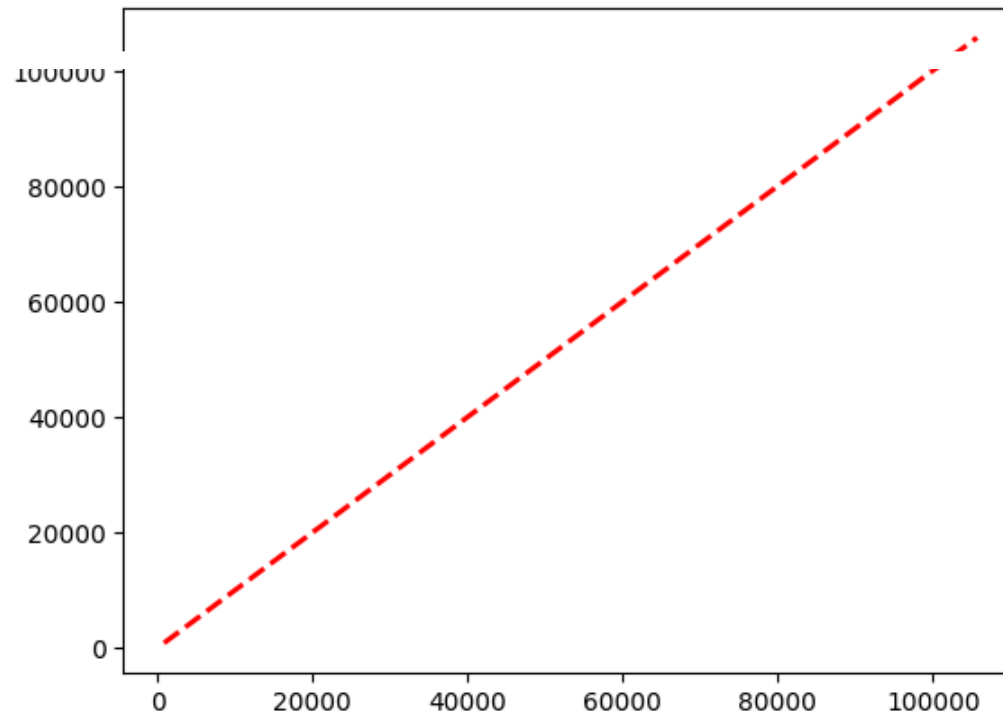
    plt.show()
```



```
plt.plot(  
    [y_test.min(), y_test.max()],  
    [y_test.min(), y_test.max()],
```

```
color='red',  
linewidth=2,  
linestyle='--'
```

```
[<matplotlib.lines.Line2D at 0x7a9a1c271430>]
```



✓ Interpretation

The scatter plot compares the actual Net Revenue values with those predicted by the Linear Regression model.

Most of the points are located close to the red diagonal line, indicating that the model's predictions closely match the actual values.

This suggests that the model has learned the relationship between the input variables and Net Revenue effectively.

The high R^2 score (approximately 0.95) further confirms that the model explains most of the variation in Net Revenue, while the relatively low MAE and RMSE values indicate small prediction errors.